GATE CS — Programming & Data Structures

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
C programming: pointers, recursion, output prediction	6.2	Tier 1
Trees, BST, heaps	2.9	Tier 1
Stacks, queues, linked lists	1.6	Tier 2

Programming & Data Structures carries about **10.7 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Programming_and_Data_Structures.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. C Programming: Pointers, Recursion, Output Prediction

Types and operators

- Typical sizes (GATE assumes 32/64-bit gcc): char 1, short 2, int 4, long 4 or 8, float 4, double 8, pointer 4 (32-bit) or 8 (64-bit). Read the question for the stated sizes.
- Integer division truncates toward zero; `%` takes the sign of the dividend ($-7 / 2 = -3$, $-7 \% 2 = -1$). Shifts: `x << k` = $x \cdot 2^k$; `x >> k` = $\text{floor}(x/2^k)$ for non-negative x.
- Precedence (high → low): `() [] -> .` → unary `! ~ ++ -- * & sizeof` → `* / %` → `+ -` → shifts → relational → `== !=` → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment → comma.
- `&&` and `||` short-circuit: in `a && b++` b is not incremented when a is 0.
- `i = i++ + ++i` style expressions are undefined behaviour; GATE normally avoids them or asks for a defined order.
- Assignment as expression: `if (x = 5)` assigns and is true. `++` / `--` as postfix return the old value.

Pointers and arrays

- `a[i] ≡ *(a+i)`. Pointer arithmetic scales by the element size: `p+1` moves sizeof(*p) bytes. `p2 − p1` gives the number of elements between them.
- An array name is a constant pointer to element 0 (not modifiable). In `sizeof(a)` an array gives total bytes, but as a function parameter it decays to a pointer.
- 2-D array `int a[R][C]` is row-major: address of a[i][j] = base + (i·C + j)·size. `*(*(a+i)+j) ≡ a[i][j]`. `a` is pointer to row; `*a` is pointer to first element.
- `int *p[3]`: array of 3 pointers. `int (*p)[3]`: pointer to an array of 3 ints. `int **pp`: pointer to pointer.
- Strings: `char s[] = "abc"` is a modifiable 4-byte array (includes '\0'). `char *s = "abc"` points to read-only literal. `strlen` excludes '\0'; `sizeof` includes it.
- C passes by value; to change a caller variable, pass its address. Swapping two ints with a function taking values does nothing.
- `void *` needs a cast before dereferencing. `malloc` returns uninitialised memory; `calloc` zeroes it; forgetting `free` is a leak; using memory after `free` is dangling-pointer undefined behaviour.

Storage classes and scope

- **static local**: initialised once, keeps its value between calls (stored in the data segment). **Global**: one copy, zero-initialised. **auto** locals: uninitialised garbage, on the stack.
- Inner-block variable with the same name *shadows* the outer one. C uses **static (lexical) scoping**: a function sees the globals where it was defined, not the caller's locals.
- Parameter passing questions (call by value, reference, name, copy-restore) appear occasionally; for pointer-based “reference”, changes persist.

Recursion

- Trace by drawing the call tree; each frame has its own locals. Count calls separately from the return value.
- Fibonacci via naive recursion: calls(n) = 1 + calls(n−1) + calls(n−2), $O(\varphi^n)$ time.
- Typical GATE recursions: GCD (`gcd(a,b) = b==0 ? a : gcd(b, a%b)`), power, binary conversion, Ackermann (grows very fast), reversing a string or list, printing before vs. after the recursive call (reverses the order).
- Stack depth for linear recursion on n is n frames: O(n) space. Tail recursion can be turned into

a loop.

> **Tip:** For output questions, make a table of variables and update it line by line. For loops with pointers, write the memory as boxes with addresses.

> **Trap:** `sizeof` on a pointer parameter, `%d` with a long, integer overflow (wraps for unsigned, undefined for signed), and operator precedence in `*p++` (value at p, then p moves) versus `(*p)++` (value at p increases).

2. Trees, BST, Heaps

Binary tree facts

- Height h (root at level 0 = height 0, a single node has height 0): max nodes 2^(h+1) − 1; min nodes h + 1 (a chain). A tree with n nodes has height between $\lfloor \log_2 n \rfloor$ and n − 1.
- In a binary tree: leaves $n_0 = n_2 + 1$, where n_2 is the number of nodes with two children. Edges = n − 1.
- Full binary tree with n internal nodes: 2n + 1 total nodes, n + 1 leaves. Complete binary tree with n nodes has height $\lfloor \log_2 n \rfloor$.
- Number of distinct binary trees with n nodes: Catalan $C_n = C(2n,n)/(n+1)$ (1, 2, 5, 14, 42). The same count of BST shapes. Labelled structures: $n! \cdot C_n$.
- Traversals: inorder (L, root, R), preorder (root, L, R), postorder (L, R, root), level order (BFS using a queue). The inorder of a BST is sorted. A tree is uniquely determined by inorder + preorder (or inorder + postorder), but not by preorder + postorder alone.
- Reconstruct: the first element of preorder (or last of postorder) is the root; find it in inorder to split left/right subtrees.
- Tree traversal with recursion uses stack space O(h); with Morris traversal O(1).

Binary search tree

- Search, insert, delete: O(h); h can be O(n) for skewed trees (e.g. inserting sorted keys), O(log n) when balanced.
- Delete: leaf → remove; one child → replace with child; two children → replace with inorder successor (min of right subtree) or inorder predecessor (max of left), then delete that node.
- Minimum = leftmost; maximum = rightmost. Inorder successor of a node with no right child is its lowest ancestor whose left subtree contains it.
- **AVL tree**: balance factor $|h(L) - h(R)| \le 1$. Min nodes $N(h) = N(h-1) + N(h-2) + 1$ with $N(0) = 1$, $N(1) = 2$ (so 1, 2, 4, 7, 12, 20, …). Rotations: LL, RR, LR, RL. Insert needs at most one rotation (single or double); delete may need O(log n).
- Red-black tree: height $\le 2 \log_2(n+1)$. B-tree: see the Databases notes.
- Number of BSTs from keys inserted in different orders: the shape depends on insertion order; n distinct keys give n! orders but only C_n shapes.

Heaps

- Binary heap = complete binary tree stored in an array. 1-indexed: parent $\lfloor i/2 \rfloor$, children 2i and 2i+1. 0-indexed: parent $\lfloor (i-1)/2 \rfloor$, children 2i+1 and 2i+2.
- Max-heap: parent ≥ children. The maximum is at the root. The minimum lies among the leaves ($\lceil n/2 \rceil$ of them, indices $\lfloor n/2 \rfloor + 1 \dots n$).
- Insert (sift-up): O(log n). Delete-max / extract: O(log n). **Build-heap** by repeated sift-down from n/2 down to 1: **O(n)**. By n repeated insertions: O(n log n).
- Number of heaps (distinct shapes) is fixed by n; number of valid max-heap arrangements of 1…n: n=3 → 2, n=4 → 3, n=5 → 8.
- Height of a heap with n nodes: $\lfloor \log_2 n \rfloor$. Nodes at height h: at most $\lceil n/2^{h+1} \rceil$.
- Heap sort: build-heap then n extractions: O(n log n) in all cases, in-place, not stable. Finding the k-th largest with a heap: O(n + k log n).
- Searching an arbitrary element in a heap takes O(n) — a heap is not a search structure.

> **Trap:** “height” can be counted in edges or in nodes. Use the definition given in the question. Also, a complete tree is not necessarily full.

3. Stacks, Queues, Linked Lists

Stack and queue operations

- Stack = LIFO (push, pop, peek); queue = FIFO (enqueue, dequeue). Both O(1) per operation with arrays or linked lists.
- Applications: expression evaluation, infix → postfix with an operator stack, parenthesis matching, recursion, DFS (stack), BFS (queue), undo.
- Postfix evaluation: scan left to right; push operands; on an operator pop two (second popped is the left operand), apply, push the result.
- Infix → postfix: operands go straight to the output; operators wait on a stack until a lower-or-equal-precedence operator arrives (left-associative); `^` is right-associative; `(` is pushed and `)` pops to the matching `(`.
- Possible output sequences of a stack: push 1…n in order; the number of valid pop sequences is Catalan C_n. A sequence is invalid if it contains a pattern like 3 … 1 … 2 (a smaller item below a larger one came out in the wrong order).
- Two stacks can implement a queue (amortised O(1) per operation); two queues can implement a stack (one operation costs O(n)).
- Circular queue with array size n: empty when front == rear; full when (rear+1) mod n == front, so capacity is n − 1 (if no counter is used).
- Deque supports insertion/deletion at both ends. Priority queue → heap.

Linked lists

- Singly linked list: insert at head O(1); insert/delete at tail O(n) without a tail pointer; delete a given node O(1) only with the previous pointer (or by copying the next node's data). Search O(n).
- Doubly linked list: O(1) delete of a known node. Circular list: last node points to the head.
- **Reverse a list** iteratively with three pointers: `next = cur->next; cur->next = prev; prev = cur; cur = next;`
- **Find the middle** with slow/fast pointers. **Cycle detection** (Floyd): fast moves 2, slow moves 1; they meet iff a cycle exists.
- Recursive list functions (swap pairs, reverse, merge): trace on a 4–5 node list. Swapping adjacent pairs of 1→2→3→4→5 gives 2→1→4→3→5.
- Merge two sorted lists O(m+n). Insertion into a sorted list O(n). Linked-list representation of a polynomial or sparse matrix saves space.
- Array vs list: arrays give O(1) random access and good cache behaviour; lists give O(1) insertion given a pointer and no resizing.

> **Tip:** For pointer-manipulation code, draw boxes and arrows and update them statement by statement; most errors come from changing a pointer before saving its old value.

4. Exam strategy & revision checklist

- Study order by marks: **C programming and recursion**, then **trees/BST/heap**, then **stacks, queues, lists**. Output-prediction questions are scoring if you trace carefully.
- Always check array bounds, integer sizes, and pointer arithmetic scaling.
- NAT answers have no negative marking: attempt every one.
- ☐ Draw the call tree of a recursive function. ☐ Reconstruct a tree from two traversals. ☐ Build a heap from an array by hand. ☐ Delete a node from a BST. ☐ Reverse a linked list on paper. ☐ Redo past questions in `GATE_CS_Programming_and_Data_Structures.pdf`.